

Assignment #2

Points: 8

Deadline for submission: 27 March 2026, 23:59

1. Objective

In **Assignment 1**, you defined a realistic system and designed a **microservices architecture consisting of 8 services and their dependencies**.

In this assignment, you will apply the **Domain-Driven Design (DDD)** approach introduced in the lectures to develop **domain models** for your system. The goal is to identify and model the **key business concepts of the domain**, their attributes, and their relationships.

Please, be aware that a **domain model represents the domain concepts and their relationships using non-technical terminology** and should not include implementation details.

1.1. System Functionalities

Before presenting the domain model, briefly list the **main functionalities of the system** derived from your proposal in Assignment 1.

These functionality statements should describe the **key operations supported by your microservices** and help explain how the domain concepts and relationships in your model were identified.

The list should remain concise and include the **main operations provided by the system**, such as creating, updating, or retrieving domain objects, as well as important domain-specific actions, including those that require interactions with other services. For example:

- R1.** The system shall allow a user to create a booking for an available trip.
- R2.** The system shall allow an administrator to update the price and available seats for a trip.
- R3.** When a user confirms a booking, the system shall process the payment, update seat availability, and store the final booking status.
- R4.** When a booking is cancelled, the system shall update the booking status, release the reserved seat, and notify the user.

...

2. Deliverable

Each team must submit **one domain model diagram representing the domain of the entire system**.

The diagram must include:

- **domain concepts** (classes)
- **attributes** of those concepts
- **relationships** between concepts

- the DDD tactical elements introduced in the lectures:
 - **Entities**
 - **Value Objects**
 - **Aggregates** (with an aggregate root)
 - **Repositories** (shown as interfaces)

The diagram should represent **business concepts only**, not software architecture.

3. Service Coverage

The domain model must represent **all microservices defined in Assignment 1**.

Each service must be represented by **at least one primary domain concept**. A **primary concept** is the main business object managed by a service (for example: *Booking*, *Payment*, *Trip*, etc.).

Each primary concept must participate in **at least one relationship** with another concept in the system.

Each concept must clearly indicate **which microservice is responsible for it** (according to your initial architecture in **Assignment 1**). For example:

Booking [BookingService]
 Payment [PaymentService]
 Customer [UserService]

A concept must belong to **only one service**. If a service requires information from another service, it should **reference the entity using an identifier** rather than redefining the concept.

The final domain model should represent **one coherent system**, not a set of isolated service models.

4. Responsibilities and Individual Contribution

Each student is responsible for **the two microservices assigned to them in Assignment 1**.

During grading, students will be evaluated **only on the concepts and relationships associated with their two services**.

The two services assigned to each student must together demonstrate the main elements of a domain model, including:

- domain concepts
- attributes
- relationships
- DDD tactical elements (entities, value objects, aggregates, repositories)

To ensure system integration, **at least one concept belonging to a student's service must have a relationship with a concept belonging to a service assigned to another team member**.

5. Expectations

Please, be aware that the goal of this assignment is to produce a **clear and coherent domain model**, not to maximize the number of concepts.

You should avoid:

- repetitive or artificial concepts
- splitting a single concept into several similar ones
- isolated service models without relationships

Important: The model should capture **the key concepts and interactions of the system** while remaining feasible to implement in the project.

Important: Domain models **must not include methods or operations**.

Important: Domain models must not include **technical infrastructure elements** like REST APIs, API gateways, authentication mechanisms, databases, message brokers, and so on.

Domain models represent business concepts only.

You can choose any diagramming tool to represent your model as far as the diagram is clear.

6. Rules for assignment #2 communication and submission

- Submit **one PDF or Word file per team** through Moodle, and the document should also contain your Names. You are organized into teams in Moodle, so only one is required to submit the assignment.
- Submit the assignment by 27 March 2026. Late submissions lose **0.5 points per hour**.
- If you have not created a **private channel for your team** in Slack until now, create one and call it after your team number (e.g., team_1, team_2, etc.), add me and the practical session supervisor to it, and use this channel **for any kind of communication.**

7. Final Remarks (Important)

Keep in mind that your next assignment as well as the project will heavily rely on this assignment.

In the project, each student will implement **two microservices**, and each service should remain **small and focused**.

In most cases, a microservice in this project will contain:

- **one or two main domain entities**
- a small number of supporting concepts
- typically, a **small set of API endpoints** (often around **5–8 per service**), depending on the service responsibilities.

Very complex domain models may become difficult to implement within the available project time. So, the goal is to design a **clear and realistic domain model**, not to maximize the number of entities or concepts.